

Lecture 11 Worksheet

บท: Recurrence Relations (cont) Introduction to Graph Theory

วิชา: 819605 Discrete Mathematics

อาจารย์ผู้สอน: ปพนธีร์ แยมโชติ | วิศวกรรมคอมพิวเตอร์ | ภาคปลาย ปีการศึกษา 2568

เป้าหมายการเรียนรู้ Week 11

1. ทบทวนการหา **closed form** ของ recurrence แบบ *plug-and-chug* จากตัวอย่างง่ายได้
2. ตั้ง recurrence ของ **Merge Sort** และ **Binary Search** แล้วเดา/พิสูจน์รูปแบบการเติบโตได้
3. เข้าใจคำศัพท์พื้นฐานของ **graph theory**: vertex, edge, degree, walk, path, cycle, connected
4. ใช้เหตุผลเชิงพิสูจน์และตัวอย่างค้านกับข้อความพื้นฐานของกราฟได้

1 Quick Review: plug-and-chug 1 ข้อ

Example 1 (ทบทวนจากคาบที่แล้ว). กำหนดให้

$$T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1).$$

จงหา closed form ของ $T(n)$ ด้วยการแทนค่าถอยกลับ เมื่อเดา closed form ได้แล้ว จงพิสูจน์ด้วย induction ว่าสูตรนั้นถูกต้องสำหรับทุก $n \in \mathbb{N}$

2 Merge Sort recurrence

2.1 โจทย์ตั้งต้น

เราจะให้ $T(n)$ แทน จำนวน comparisons มากที่สุด ที่ Merge Sort ใช้ในการเรียงข้อมูล n ตัว และเพื่อให้ช้ง่าย เรา จะสมมติว่า n เป็นกำลังของ 2

ภาพรวมของ Merge Sort

ทบทวนแนวคิด Merge Sort:

1. ถ้าลิสต์มีขนาด 1 ก็ sorted อยู่แล้ว
2. ถ้าลิสต์มีขนาดมากกว่า 1 ให้แบ่งออกเป็นสองครึ่ง
3. เรียงแต่ละครึ่งแบบ recursion
4. นำสองลิสต์ที่ sorted แล้วมา merge กลับเข้าด้วยกัน

2.2 ตั้ง recurrence

Example 2 (Merge step). อธิบายว่าทำไมการ merge ลิสต์ sorted สองลิสต์ที่มีสมาชิกทั้งหมดรวมกัน n ตัว ใช้ comparisons มากที่สุดไม่เกิน $n - 1$ ครั้ง และทำไม recurrence relation คือ $T(n) = 2T(n/2) + (n - 1)$ โดย $T(1) = 0$

2.3 Plug-and-chug

Example 3. จงแทนค่าถอยกลับเพื่อหา pattern ของ $T(n)$:

$$T(n) = 2T(n/2) + (n - 1), \quad T(1) = 0$$

คำแนะนำ: ใช้ $n = 2^k$

3 Binary Search recurrence

Pseudocode: Recursive Binary Search

```
RBSearch( $A, x, \ell, r$ ):  
    if  $\ell > r$  then return 0  
     $m \leftarrow \lfloor (\ell + r)/2 \rfloor$   
    if  $A[m] = x$  then return  $m$   
    if  $x < A[m]$  then return RBSearch( $A, x, \ell, m - 1$ )  
    else return RBSearch( $A, x, m + 1, r$ )
```

Example 4. หา 5 จาก sorted array $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]$

3.1 การวิเคราะห์

“ในกรณีแย่ที่สุด Binary Search ใช้การเปรียบเทียบกี่ครั้ง?”

ตั้งฟังก์ชันต้นทุน

ให้ $B(n)$ แทนจำนวน comparisons มากที่สุดที่ Binary Search ใช้กับลิสต์ sorted ขนาด n เพื่อให้ง่าย เราจะเริ่มจากกรณีที่ n เป็นกำลังของ 2 และใช้วิธีตัดแบ่งครึ่งเลย

Example 5. เพราะเหตุใด recurrence ของ Binary Search จึงควรเป็น

$$B(1) = 1, \quad B(n) = B(n/2) + 1 \quad (n \geq 2)?$$

3.2 Plug-and-chug

Example 6. จงแทนค่าถอยกลับ:

$$B(n) = B(n/2) + 1$$

สิ่งที่ควรได้

ถ้า $n = 2^k$ จะได้

$$B(n) = \log_2 n + 1.$$

ดังนั้น Binary Search โตรระดับ

$$B(n) = \Theta(\log n).$$

Exercise 7. เปรียบเทียบ Merge Sort กับ Binary Search:

1. เหตุใดทั้งสองตัวอย่างจึงเป็น recursion เหมือนกัน แต่ได้อัตราการเติบโตไม่เหมือนกัน
2. ใน recurrence ของ Merge Sort มีส่วนใดที่ ``แพงเพิ่ม" เมื่อเทียบกับ Binary Search

4 Introduction to Graph Theory (สำรองเพื่อสอนทัน)

4.1 นิยามพื้นฐาน

Definition 8 (simple graph). graph แบบง่าย (simple graph) คือคู่อันดับ

$$G = (V, E)$$

โดยที่

- V คือเซตของ **vertices** (จุดยอด)
- E คือเซตของ **edges** (เส้นเชื่อม) ซึ่งแต่ละเส้นเชื่อมเชื่อม **vertex** สองตัวที่ต่างกัน

และใน simple graph จะไม่มี self-loop และไม่มีเส้นเชื่อมซ้ำ (กล่าวคือสนใจแค่คอนเซปต์การเชื่อมหรือไม่เชื่อมของจุด)

Definition 9 (degree). ดีกรีของ vertex v เขียนเป็น $\deg(v)$ คือจำนวนเส้นเชื่อมที่ incident กับ v

Example 10. กำหนด

$$V = \{a, b, c, d, e\}, \quad E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}, \{d, e\}\}.$$

1. วาดกราฟนี้
2. หา $\deg(a), \deg(b), \deg(c), \deg(d), \deg(e)$
3. มี cycle หรือไม่

4.2 Handshake Lemma

Theorem 11 (Handshake Lemma). สำหรับ simple graph ใด ๆ

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Example 12. จงพิสูจน์ Handshake Lemma

แนวคิด: Double Counting

ให้นับ ``จำนวนปลายเส้นเชื่อมทั้งหมด" สองวิธี (1) นับทีละ vertex (2) นับทีละ edge

Exercise 13 (ผลตามของ Handshake Lemma). จงพิสูจน์ว่าใน simple graph ใด ๆ จำนวนของ vertex ที่มีดีกรีคี่ต้องเป็นจำนวนคู่

4.3 Walk, Path, Cycle, Connected

Definition 14. (เวอร์ชันเชิงแนวคิด) ให้ G เป็น graph

- **walk** คือการเดินตาม edges ต่อกันได้
- **path** คือ walk ที่ไม่ผ่าน vertex ซ้ำ
- **cycle** คือ path แบบปิดที่กลับมาจุดเริ่มต้น
- graph เรียกว่า กราฟต่อเนื่อง (**connected**) ถ้าสำหรับทุกคู่ของ vertices มี path เชื่อมถึงกัน

Example 15. พิจารณา graph H ที่มี

$$V(H) = \{a, b, c, d, e, f\},$$

$$E(H) = \{\{a, b\}, \{b, c\}, \{c, a\}, \{c, d\}, \{e, f\}\}.$$

ตอบคำถามต่อไปนี้:

1. ยกตัวอย่าง walk จาก a ไป d
2. ยกตัวอย่าง path จาก a ไป d
3. ยกตัวอย่าง cycle
4. graph นี้ connected หรือไม่
5. graph นี้มี connected component กี่ส่วน

ลองนึกด้วยตัวเองดูก่อนว่าเราจะนิยามสิ่งต่าง ๆ เหล่านี้ด้วยเงื่อนไขอย่างไร เพราะที่นิยามไว้ตอนนี้เป็นแค่คำนิยามตามแนวคิดเชิงรูปภาพ แต่เอาไปพิสูจน์อะไรต่อไม่ได้ ถ้าจะนำไปพิสูจน์ก็ต้องทำให้ formal (ถ้าเปิดหาหรือถามอะไรมาจะได้ฝึกในการนิยามของตัวเอง)

Example 16. จงพิสูจน์(ด้วย induction)ว่า

ถ้า $G = (V, E)$ เป็นกราฟต่อเนื่อง แล้ว $|V| \leq |E| + 1$